MediCare HMS

Hospital Management System — Project Documentation

Phase 1 & 2 Complete

A full-stack hospital management system built with Next.js 15, NestJS, PostgreSQL, and TypeORM. Covers patient management, appointments, clinical workflows, pharmacy, billing, and role-based access control.

9	14	26	73
User Roles	Backend Modules	DB Entities	API Endpoints

42	13	2	15
Frontend Pages	UI Components	Languages	Active Pages

Tech Stack: Next.js 15 · React 19 · NestJS 11 · PostgreSQL · TypeORM · Tailwind v4 · Zustand · React Query v5

Generated: July 2026

Table of Contents

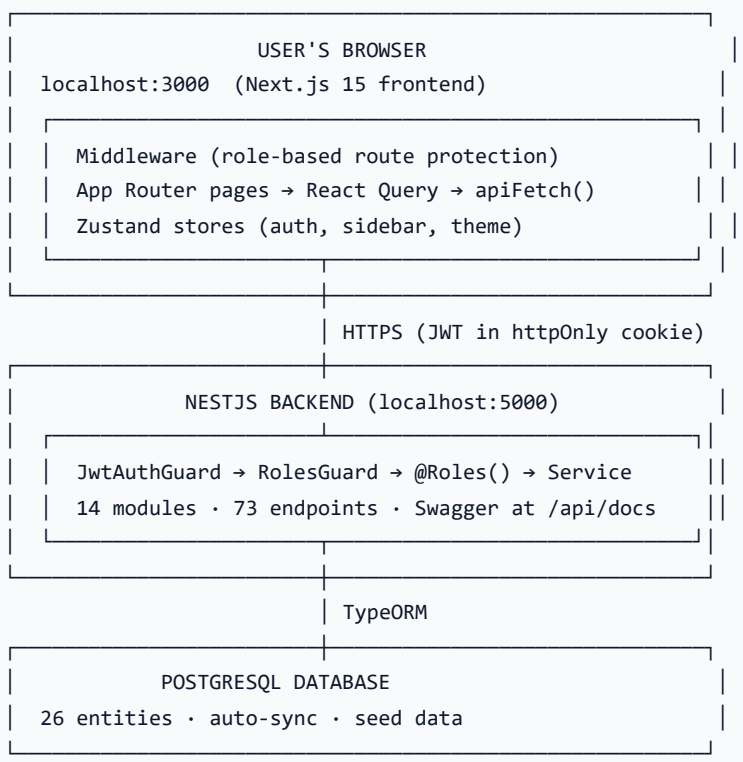
1. Project Overview

MediCare HMS is a comprehensive hospital management system designed for healthcare facilities in Bangladesh. It digitizes the entire patient journey — from appointment booking and registration, through clinical consultations, vitals recording, prescriptions, lab tests, pharmacy dispensing, billing, and inpatient admissions.

15 fully functional pages with real data fetching, plus 9 reusable UI components, role-based access control with data ownership enforcement, bilingual support (English/Bengali), and a public TV display screen for waiting rooms.

Area	Status	Details
Authentication	Complete	JWT + httpOnly cookies, refresh token rotation, account lockout
Patient Management	Complete	Registration, search, detail with 7 tabs, full medical history
Appointments	Complete	4-step booking wizard, slot locking, queue management (Kanban)
Doctor Schedules	Complete	Weekly schedule grid, slot generation, availability
Vitals Recording	Complete	Nurse dashboard, 8 vitals fields, auto BMI, clinical alerts
Role-Based Access	Complete	9 roles, ownership-scoped queries, frontend + backend enforcement
Backend APIs	Complete	14 modules, 73 endpoints, Swagger documented
Frontend UI	Complete	15 active pages, 9 UI components, skeletons, empty states, dark mode
Billing / Pharmacy / Lab / Prescriptions	Backend Ready	APIs built, frontend pages are placeholders
Reports / Settings / Notifications	Backend Ready	APIs built, frontend pages are placeholders

2. System Architecture



Project Structure

```
HMS/
├── backend/                    # NestJS API (port 5000)
│   ├── src/modules/            # 14 feature modules
│   ├── src/entities/           # 26 TypeORM entities
│   ├── src/common/             # Guards, decorators
│   └── src/seeds/              # Demo data seeder
├── frontend/                   # Next.js 15 App (port 3000)
│   ├── src/app/                # 42 pages (App Router)
│   ├── src/components/         # 13 components (9 UI primitives)
│   ├── src/hooks/              # React Query hooks
│   ├── src/stores/             # Zustand stores
│   └── src/i18n/               # English + Bengali
├── shared/                     # Shared types
├── start.bat                   # Windows quick-start script
├── start.sh                    # Linux/Mac quick-start script
└── package.json                # Workspace root
```

3. Technology Stack

Layer	Technology	Version	Purpose
Frontend Framework	Next.js (App Router)	15	SSR/SSG, routing, React Server Components
UI Library	React	19	Component rendering
Styling	Tailwind CSS	v4	Utility-first CSS with @theme tokens
State Management	Zustand	5	Auth, sidebar, theme stores (persisted)
Data Fetching	TanStack React Query	5	Server state, caching, auto-refetch
Forms	React Hook Form + Zod	7 / 3	Form validation
Icons	lucide-react	0.468	SVG icon set
Toasts	sonner	1.7	Toast notifications
i18n	next-intl	3.26	English + Bengali translations
Backend Framework	NestJS	11	Modular Node.js framework
ORM	TypeORM	0.3	PostgreSQL entity mapping
Database	PostgreSQL	16	Relational database
Auth	Passport-JWT	4	JWT strategy + refresh tokens
API Docs	Swagger (OpenAPI)	11	Auto-generated API docs
Validation	class-validator	—	DTO validation
Security	Helmet, bcrypt, Throttler	—	HTTP headers, password hashing, rate limiting

4. User Roles & Access Control

The system has **9 distinct roles**, each with different permissions. Access is enforced at four layers:

1. **Frontend middleware** — checks JWT role against route rules, redirects unauthorized users
2. **Frontend dashboard layout** — client-side role-gate mirroring middleware
3. **Backend RolesGuard** — checks `@Roles()` decorator on every endpoint
4. **Backend ownership scoping** — service queries auto-filter by `staffId` or `patientId` from JWT

Super Admin

Full system access — no restrictions

Email: `admin@medicare.com` **Password:** `Admin@123`

- ✅ Can view and manage everything in the system
- ✅ Access to all dashboards, all modules, all data
- ✅ Can manage staff, settings, reports, audit logs
- ✅ Can edit any doctor's schedule, discharge any patient
- 📍 Lands on: `/admin` dashboard

Admin

Full system access (same as Super Admin)

Email: `admin@medicare.com` (shared in demo)

- ✅ Same permissions as Super Admin — all modules accessible
- ✅ Can manage staff, billing, patients, appointments
- 📍 Lands on: `/admin` dashboard

Doctor

Sees only their own patients & appointments

Email: `doctor@medicare.com` **Password:** `Doctor@123`

Data privacy enforced: A doctor can only see patients they have appointments with. They cannot view other doctors' appointments, prescriptions, or patients.

- ✅ View own appointments (filtered by `doctorId` automatically)
- ✅ View patients they have treated (relationship check)
- ✅ Create prescriptions for own patients
- ✅ Request lab tests for own patients
- ✅ View vitals for own appointments
- ✅ Edit own weekly schedule only
- ✅ Discharge own admitted patients
- 🔒 Cannot see other doctors' patients or appointments
- 📍 Lands on: `/doctor` dashboard

Nurse

Ward-level access — vitals recording & queue management

Email: `nurse@medicare.com` **Password:** `Nurse@123`

- ✅ View today's appointment queue (all patients, ward-level)
- ✅ Record patient vitals (temperature, BP, pulse, SpO2, weight, height, BMI auto-calc)
- ✅ View vitals for any appointment
- ✅ View all patients (ward-level access)
- ✅ Admit patients, transfer beds
- ✅ Add progress notes to admissions
- ✅ Clinical alerts (hypertensive crisis, hyperpyrexia, hypoxemia)
- 📍 Lands on: `/nurse` dashboard

Receptionist

Patient registration & appointment management

Email: `reception@medicare.com` **Password:** `Reception@123`

- ✅ Register new patients
- ✅ Search and view all patients
- ✅ Book appointments (4-step wizard)
- ✅ Manage appointment queue (check-in, start visit, complete, cancel)
- ✅ View all appointments
- ✅ Admit patients
- ✅ Create invoices and process payments
- 📍 Lands on: `/receptionist` dashboard

Pharmacist

Medicine inventory & dispensing

Email: `pharmacist@medicare.com` **Password:** `Pharmacist@123`

- ✅ View medicine inventory
- ✅ Add, update medicines
- ✅ Adjust stock levels
- ✅ View low-stock alerts
- ✅ Dispense medicines
- 🔒 Cannot access patient medical records
- 📍 Lands on: `/pharmacy` dashboard

Lab Technician

Lab test processing — all tests visible for processing

Email: `lab@medicare.com` **Password:** `Lab@123`

- ✅ View all lab tests (for processing)
- ✅ Collect samples
- ✅ Start processing tests
- ✅ Complete tests with results
- ✅ View lab test types catalog
- 🔒 Cannot access patient medical records beyond lab tests
- 📍 Lands on: `/lab` dashboard

Accountant

Billing, invoices & financial reports

Email: `accountant@medicare.com` **Password:** `Accountant@123`

- ✅ View all invoices and payments
- ✅ Create invoices
- ✅ Process payments
- ✅ View daily collection reports
- ✅ View outstanding bills
- 🔒 Cannot access medical records (prescriptions, vitals, lab results)
- 📍 Lands on: `/accountant` dashboard

🧑 Patient

Self-service portal — sees only their own data

Data privacy enforced: A patient can only see their own appointments, bills, lab reports, and prescriptions. All queries are auto-filtered by `patientId` from the JWT.

✅ View own appointments

✅ View own prescriptions

✅ View own lab reports

✅ View own invoices/bills

🔒 Cannot see any other patient's data — enforced at database query level

📍 Lands on: `/portal` dashboard

5. Features by Module

5.1 Authentication & Security

Feature	Description
JWT + httpOnly cookies	Access token (15 min) in httpOnly cookie; refresh token (7 days) with path restriction
Refresh token rotation	Each refresh generates a new token; old one revoked. Token reuse detection revokes entire family
Account lockout	After 5 failed login attempts, account is locked for 15 minutes
Bcrypt password hashing	12 rounds — passwords never stored in plaintext
Helmet security headers	XSS protection, content-type sniffing prevention, etc.
Rate limiting (Throttler)	Prevents brute-force attacks on auth endpoints
Role-based access (RBAC)	`@Roles()`

6. Backend API Endpoints

Base URL: `http://localhost:5000/api/v1` | Swagger docs: `http://localhost:5000/api/docs`

All endpoints require JWT unless marked **Public**.

Method	Endpoint	Roles	Description
Authentication			
POST	`/auth/login`	**Public**	Login, returns JWT in cookies
POST	`/auth/refresh`	**Public**	Refresh access token
POST	`/auth/logout`	Authenticated	Revoke tokens, clear cookies
GET	`/auth/me`	Authenticated	Get current user profile
Patients			
GET	`/patients`	Admin, Rec, Doctor, Nurse	List with search, filters, pagination
POST	`/patients`	Admin, Rec	Register new patient
GET	`/patients/:id`	Admin, Rec, Doctor, Nurse	Full patient record (ownership checked)
PUT	`/patients/:id`	Admin, Rec, Doctor	Update patient info
DELETE	`/patients/:id`	Admin	Archive (soft-delete)
GET	`/patients/:id/history`	Admin, Rec, Doctor, Nurse	Aggregated medical history
GET	`/patients/:id/appointments`	Admin, Rec, Doctor, Nurse	Patient appointment timeline
GET	`/patients/:id/prescriptions`	Admin, Rec, Doctor, Nurse	Patient prescriptions
GET	`/patients/:id/lab-tests`	Admin, Rec, Doctor, Nurse	Patient lab tests

Appointments			
GET	`/appointments`	All staff + Patient	List (auto-scoped by role)
POST	`/appointments`	Admin, Rec, Patient	Book appointment (slot locking)
GET	`/appointments/today-queue`	All staff	Today's queue grouped by status
GET	`/appointments/:id`	All staff + Patient	Details (ownership checked)
PUT	`/appointments/:id/status`	All staff	Update status
Schedules			
GET	`/schedules/doctor/:id`	Authenticated	Doctor's weekly schedule
PUT	`/schedules/doctor/:id`	Admin, Doctor	Set schedule (own only)
GET	`/schedules/available`	Authenticated	Available slots for a doctor
Vitals			
POST	`/appointments/:id/vitals`	Admin, Nurse	Record vitals
GET	`/appointments/:id/vitals`	Admin, Doctor, Nurse	Get vitals (doctor ownership checked)
Prescriptions			
GET	`/prescriptions`	Admin, Doctor, Nurse, Patient	List (auto-scoped by role)
POST	`/prescriptions`	Admin, Doctor	Create prescription
GET	`/prescriptions/:id`	Admin, Doctor, Nurse, Patient	Details (ownership checked)
Lab Tests			
GET	`/lab-tests`	Admin, Doctor, Lab, Patient	List (auto-scoped by role)
POST	`/lab-tests`	Admin, Doctor	Request a lab test
GET	`/lab-tests/:id`	Admin, Doctor, Lab, Patient	Details (ownership checked)

PUT	`/lab-tests/:id/collect`	Admin, Lab	Mark sample collected
PUT	`/lab-tests/:id/complete`	Admin, Lab	Complete with results
Admissions			
GET	`/admissions`	Admin, Rec, Nurse, Doctor	List (doctor scoped)
POST	`/admissions`	Admin, Rec, Nurse	Admit patient
GET	`/admissions/:id`	Admin, Rec, Nurse, Doctor	Details (ownership checked)
PUT	`/admissions/:id/discharge`	Admin, Doctor	Discharge (ownership checked)
Billing			
GET	`/billing/invoices`	Admin, Rec, Accountant, Patient	List (patient scoped)
POST	`/billing/invoices`	Admin, Rec, Accountant	Create invoice
GET	`/billing/invoices/:id`	Admin, Rec, Accountant, Patient	Details (patient scoped)
POST	`/billing/invoices/:id/payments`	Admin, Rec, Accountant	Add payment
Staff & Departments			
GET	`/staff`	Admin	List all staff
GET	`/staff/doctors`	Authenticated	List doctors (for booking)
GET	`/departments`	Authenticated	List departments

7. Frontend Pages

15 pages are fully functional with real data. 27 pages are polished placeholders.

Active Pages (with real data)

Page	Route	Roles	Features
Login	`/login`	Public	Gradient background, demo account quick-fill buttons
Admin Dashboard	`/admin`	Admin	6 stat cards, revenue/activity panels
Doctor Dashboard	`/doctor`	Doctor	4 quick stats, today's schedule placeholder
Nurse Dashboard	`/nurse`	Nurse	Pending vitals queue + vitals form, clinical alerts
Patients List	`/patients`	Admin, Rec, Doctor	Search, filters, pagination, skeleton loading
Patient Detail	`/patients/:id`	Admin, Rec, Doctor, Nurse	7-tab interface, summary cards, medical history
New Patient	`/patients/new`	Admin, Rec	Registration form
Appointments List	`/appointments`	Admin, Rec, Doctor, Nurse	Date filter, status badges, table
Book Appointment	`/appointments/new`	Admin, Rec	4-step wizard: patient → doctor → time → confirm
Queue Management	`/appointments/queue`	Admin, Rec, Doctor, Nurse	Kanban board, 4 status columns, action buttons
Schedules	`/schedules`	Admin, Doctor	Doctor selector, 7-day calendar grid
Patient Portal	`/portal`	Patient	Welcome card, 5 menu cards

Portal: Appointments	`/portal/appointments`	Patient	Own appointments (auto-scoped)
Portal: Bills	`/portal/bills`	Patient	Own invoices (auto-scoped)
Portal: Lab Reports	`/portal/lab-reports`	Patient	Own lab tests (auto-scoped)
Portal: Prescriptions	`/portal/prescriptions`	Patient	Own prescriptions (auto-scoped)
TV Display	`/display/queue`	**Public**	Full-screen Now Serving + next 5, live clock, auto-refresh

Placeholder Pages (Backend Ready)

Page	Route	What's Coming
Receptionist	`/receptionist`	Patient registration, appointment booking, queue management
Pharmacy	`/pharmacy`	Inventory, dispensing, stock alerts
Lab	`/lab`	Test requests, sample collection, results
Billing	`/billing`	Invoices, payments, collection reports
Prescriptions	`/prescriptions`	Create prescriptions, medicine search
Staff	`/staff`	Staff directory, roles, departments
Admissions	`/admissions`	Patient admission, bed management, discharge
Accountant	`/accountant`	Revenue tracking, expense management
Reports	`/reports`	Analytics, patient stats, financial reports
Settings	`/settings`	Hospital settings, user management

8. Data Privacy and Security

The system enforces data privacy at **four layers.**

Layer 1: Frontend Middleware

Next.js middleware extracts JWT role from cookie, checks against 16 route-to-role patterns, redirects unauthorized users.

Layer 2: Frontend Dashboard Layout

Client-side mirror of the middleware.

Layer 3: Backend RolesGuard

Every endpoint has @Roles() checked by RolesGuard. Throws ForbiddenException if role not allowed.

Layer 4: Backend Ownership Scoping

Role	What they see	How it's enforced
Doctor	Only their own appointments, patients, prescriptions, lab requests	WHERE doctorId = staffId; findOne checks ownership
Patient	Only their own appointments, bills, lab reports, prescriptions	WHERE patientId = patientId from JWT
Nurse	All patients (ward-level)	No patient filter
Receptionist	All patients and appointments	No filter
Pharmacist	Medicine inventory only	@Roles restricts
Lab Tech	All lab tests for processing	@Roles restricts
Accountant	Billing/invoices only	@Roles restricts
Admin	Everything	No filter

Result: A doctor can only see their own appointments and patients they have treated. Even if they guess another patient UUID, the backend returns 403. A patient can only see their own data.

9. Default Login Credentials

Role	Email	Password	Landing
Super Admin	`admin@medicare.com`	`Admin@123`	`/admin`
Doctor	`doctor@medicare.com`	`Doctor@123`	`/doctor`
Receptionist	`reception@medicare.com`	`Reception@123`	`/receptionist`
Nurse	`nurse@medicare.com`	`Nurse@123`	`/nurse`
Pharmacist	`pharmacist@medicare.com`	`Pharmacist@123`	`/pharmacy`
Lab Technician	`lab@medicare.com`	`Lab@123`	`/lab`
Accountant	`accountant@medicare.com`	`Accountant@123`	`/accountant`

15 demo doctors: all with password `Doctor@123`, email pattern `dr.firstname.lastname@medicare.com`

10. How to Start

Windows: Double-click `start.bat`
Linux/Mac: Run `./start.sh`
Starts backend (port 5000) + frontend (port 3000) concurrently.

1. Install: `npm install`
2. Database: Create PostgreSQL db `medicare_hms` (user: postgres / pass: 1234)
3. Seed: `npm run seed`
4. Start: `npm run dev`
5. Open: `http://localhost:3000`

11. Future Phases

Phase 3 — Frontend Page Implementation

Module	Backend	Frontend Work
Billing	Ready	Invoice list, create form, payments, collection report
Pharmacy	Ready	Medicine inventory, stock adjustment, dispense form
Lab Tests	Ready	Test request list, status workflow, result entry
Prescriptions	Ready	Prescription list, create form, print/export
Admissions	Ready	Admission list, bed/ward board, discharge form
Staff	Ready	Staff directory, role management
Reports	Ready	Dashboard with Recharts, export to PDF/Excel

Phase 4 — Enhancements

- Charts and data visualization
- Mobile-responsive sidebar drawer
- Real-time notifications (WebSocket)
- PDF export for invoices, prescriptions
- Full dark mode across all pages
- Accessibility improvements

Phase 5 — Deployment

- Dockerfile + docker-compose
- CI/CD pipeline
- Jest unit tests + e2e tests

MediCare HMS

Phase 1 and 2 Documentation — Complete

Built with Next.js 15, NestJS 11, PostgreSQL, TypeORM
Tailwind CSS v4, Zustand, React Query v5
Generated July 2026